

# FitBuddy: Complete Development Guide

AI Fitness Plan Generator using Gemini Models | Full Project Roadmap & Implementation

## 1. Project Overview & Architecture

**FitBuddy** is an end-to-end AI-powered fitness application designed to generate personalized workout and nutrition plans. It utilizes Google's advanced Generative AI (**Gemini Models**) on the backend, structured through **FastAPI**, and rendered dynamically to users via a responsive **HTML/CSS/Jinja2** frontend.

**Architecture Summary:** FastAPI handles server routing and request/response lifecycles, google-generativeai SDK processes user inputs to generate tailored health content, and Jinja2 template engine delivers dynamic pages.

## 2. Development Phases (Epics & Stories Roadmap)

- **Epic 1: Setup & Architecture**
  - Research and Select Appropriate Generative AI Model (Gemini Pro)
  - Define system architecture and secure environment configuration (.env)
  - Set up virtual environment and Python dependencies (**fastapi**, **uvicorn**, **google-generativeai**)
- **Epic 2: Core Functionalities Development**
  - Implement backend logic to handle prompt engineering for workout plans
  - Integrate FastAPI backend to manage user input processing and routing
- **Epic 3: App.py Development**
  - Initialize the FastAPI application instance and configure static files/templates
  - Set up API key configuration and connect modular route files (**routes.py**)
- **Epic 4: Frontend Development**
  - Design user-friendly input forms (**index.html**) for age, weight, goals, and dietary preferences
  - Create responsive layouts and styled dashboards (**style.css**) using Jinja2 templates
- **Epic 5: Deployment & Verification**
  - Prepare application for local execution and test Uvicorn server (**uvicorn app:app --reload**)
  - Verify end-to-end response rendering and debug edge cases

## 3. Key Implementation Code Snippets

### A. Backend Entry Point (*app.py*)

```
import os
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from dotenv import load_dotenv
import google.generativeai as genai

load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

app = FastAPI(title="FitBuddy - AI Fitness Generator", version="1.0.0")
app.mount("/static", StaticFiles(directory="static"), name="static")
templates = Jinja2Templates(directory="templates")

from routes import router as main_router
app.include_router(main_router)

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("app:app", host="127.0.0.1", port=8000, reload=True)
```

### B. Gemini AI Integration & Routing (*routes.py*)

```
from fastapi import APIRouter, Request, Form
from fastapi.templating import Jinja2Templates
import google.generativeai as genai

router = APIRouter()
templates = Jinja2Templates(directory="templates")

@router.post("/generate")
async def generate_plan(request: Request, age: int = Form(...), goal: str = Form(...)):
    model = genai.GenerativeModel('gemini-pro')
    prompt = f"Create a 7-day fitness and diet plan for a {age} year old person whose goal is {goal}."
    response = model.generate_content(prompt)
    return templates.TemplateResponse("plan.html", {"request": request, "plan": response.text})
```

## 4. Conclusion & Execution

By systematically executing each Epic, *FitBuddy* delivers a seamless, AI-driven wellness platform. Launch the server locally by running `uvicorn app:app --reload` and navigating to <http://127.0.0.1:8000>.